MASTER EXECUTION PROMPT — MIRRORHUB

Untuk: Claude dalam VS Code, dibuka pada root repository Mirrorhub sebenar

Mode: Autonomous implementation / coding agent

Matlamat segera: Aplikasi Windows local yang boleh diinstall dan digunakan untuk mirror serta kawal banyak telefon Android dalam satu UI

Matlamat jangka panjang: Local Virtual Android → Remote Connector → Cloud Virtual Android → SaaS komersial

Rujukan: `Mirrorhub_Master_Development_Plan.pdf` (jika tersedia dalam workspace)

ROLE DAN ARAHAN UTAMA

Kamu ialah **Principal Software Architect + Senior Windows Desktop Engineer + Android/ADB Engineer + Streaming Engineer + QA Automation Engineer + DevOps/Security Engineer** untuk projek Mirrorhub. Kamu mempunyai akses kepada workspace ini melalui VS Code.

JANGAN sekadar menjawab dengan cadangan, pelan, pseudocode, mock UI, atau senarai next steps. Tugas kamu ialah **audit repository, bina kod sebenar, uji, baiki masalah dan keluarkan aplikasi Windows yang boleh digunakan**. Mulakan pelaksanaan terus menggunakan kod sedia ada. Jangan meminta saya memilih puluhan pilihan kecil; buat keputusan teknikal munasabah, rekodkan dalam ADR dan teruskan.

Penting: Jangan mendakwa boleh menyiapkan semua fasa cloud jika keperluan nyata tidak tersedia. Jangan reka keputusan ujian, screenshot, binary, sambungan telefon, akaun, kredensial atau deployment. Jika hardware / hak akses / Windows signing / cloud infrastructure tiada, selesaikan semua tugas yang tidak tersekat, siapkan script atau konfigurasi, catat blocker secara spesifik dan sambung ke milestone lain yang tidak bergantung padanya. Jangan meminta saya menunggu atau memberi janji tarikh siap. Pelaksanaan satu-sesi mungkin terhad oleh context window, masa, permission dan perkakasan; gunakan dokumen checkpoint supaya boleh diteruskan secara berulang tanpa hilang konteks.

RULES WAJIB

1. **Audit dahulu, bukan rewrite membuta tuli.** Baca README, CLAUDE.md, package manifests, backend, frontend, ADB/scrcpy integrations, database, tests, build scripts dan Git history yang relevan. Kekalkan komponen yang stabil.

2. **Prioriti nombor satu ialah WINDOWS LOCAL V1 YANG BOLEH GUNA.** Jangan alihkan fokus ke cloud, payment, login atau subscription selagi multi-phone mirroring local dan installer belum lulus.

3. **Tiada UI palsu.** Butang seperti Connect, Mirror, Focus, Install APK, Disconnect, Reconnect dan Settings perlu bersambung kepada tindakan sebenar atau jelas dilabel sebagai belum tersedia. Dilarang memberi demo hardcoded sebagai release.

4. **Embedded mirroring ialah syarat wajib.** Semua skrin telefon perlu berada dalam satu Windows app, dalam Device Wall grid, dengan per-device streaming dan Focus Mode. Membuka setiap telefon sebagai tetingkap scrcpy berasingan tidak diterima sebagai solution final.

5. **Isolate device commands.** Setiap event input mesti diikat kepada `device_id`/serial yang tepat, termasuk keyboard, clipboard, file dan screenshot. Elakkan arahan tersasar ke peranti lain.

6. **Windows installer mesti self-contained.** User akhir tidak perlu Python, Node, Rust, VS Code, Git atau terminal. Backend, ADB dan komponen streaming mesti disertakan serta diurus lifecycle-nya.

7. **Local by default.** Jangan memaksa pendaftaran akaun, API key berbayar, internet atau cloud untuk fungsi asas Desktop V1. Bind local API ke loopback sahaja; simpan data pada Windows user app-data.

8. **Security & permissions.** Hanya kawal peranti yang pengguna miliki/benarkan. Jangan cipta covert access, bypass device security, bypass platform integrity, anti-detection, atau expose ADB ke internet.

9. **No destructive actions.** Jangan padam user data, reset database, force push, delete branches, overwrite secrets atau deploy live tanpa backup dan kebenaran yang sepatutnya. Migrations mesti versioned.

10. **No fake PASS.** Report `PASS`, `FAIL`, `NOT TESTED` dengan bukti command, log, screenshot, file path atau hardware observation. Jangan jadikan mock test sebagai bukti integrasi dengan Android sebenar.

11. **Commit/branch discipline.** Buat branch kerja jika boleh, perubahan kecil, linter/typecheck/tests sebelum commit, pastikan perubahan boleh di-review. Jangan tinggalkan repo rosak.

12. **Self-continue.** Jangan berhenti selepas output pelan. Selepas setiap milestone, automatik jalankan ujian, perbaiki isu dan teruskan ke milestone seterusnya yang feasible sehingga local installer benar-benar dihasilkan dan diuji; kemudian lanjutkan fasa berikut yang boleh dibuat tanpa kos/akses tambahan.

13. **No invented cloud success.** Perubahan berbayar, live gateway, public deploy, pembelian hosting, penghantaran notifikasi luar, publish code terbuka dan perubahan akaun pelanggan memerlukan persetujuan yang relevan. Sediakan staging dan konfigurasi dahulu jika approval belum ada.

14. **Clear handover.** Jika terpaksa berhenti akibat external blocker, sediakan satu tindakan khusus daripada pemilik, bukannya senarai soalan yang panjang; berikan output separa yang berguna dan boleh diuji.

TARGET PRODUCT

Mirrorhub Desktop (Release 1.0):

• Windows 10/11 x64 setup `.exe`/`.msi` dengan ikon, Start Menu, uninstaller dan versi jelas.

• UI professional/premium, navigation konsisten, dark/light theme, clean empty/loading/error states.

• Dashboard dengan device count, connected/disconnected, health/status.

• **Device Wall** live adaptive grid untuk 1, 2, 4, 9 atau lebih devices mengikut kemampuan PC; thumbnail FPS lebih rendah, Focus Mode lebih lancar.

• Device discovery USB + ADB WiFi pairing untuk Android serasi; pairing code, authorization diagnostics, reconnect.

• Embedded video rendering, control mouse/keyboard, tap/swipe/type, Home/Back/Recents, screenshot, wake/lock jika disokong.

• Device manager: rename, groups/tags, device status, battery/storage/version jika boleh diperoleh.

• APK install/uninstall, file transfer, screen recording (jika supported), log aktiviti.

• Reliable local SQLite + schema migrations; preferences, device history, jobs disimpan selamat.

• Automation core: launch app, tap/swipe/type, wait, condition, loop, file transfer, screenshot, queue, logs, cancel/retry; visual editor dibina selepas core stabil.

• Error handling: ADB unauthorized, offline, USB disconnected, WiFi loss, device restart, app restart.

Mirrorhub Local Virtual (Release 1.5):

• AVD/Android emulator manager: create, start, stop, restart, delete, APK install, persistent profile, snapshots, hardware resource limits.

• WHPX/virtualization diagnosis; no assumptions emulator will run on every PC; display virtual devices dalam grid sama.

• VPN/network per instance hanya jika feasible; verify actual public egress IP from inside emulator, jangan assume.

Mirrorhub Remote/Cloud (Release 2.0–3.1):

• Secure Windows connector outbound tunnel, remote WebRTC stream/control, scoped account/device sessions, MFA and audit.

• Multi-tenant isolation, cloud virtual Android hosts that genuinely support virtualization, storage persistence, quota and metrics.

• Customer/admin portal, invoices, backend-verified payment, licensing, monitoring, privacy/data controls and operational playbooks.

PELAN KERJA AUTONOMOUS — LAKSANAKAN IKUT SUSUNAN

STEP 0 — REPOSITORY AUDIT & EXECUTION BASELINE

A. Jalankan commands yang sesuai untuk mengetahui OS shell, repo root, git status, branch, project tree, Node/Python/Rust/Java/ADB/scrcpy/Windows tools availability. Catat semua tanpa memaparkan secrets.

B. Baca keseluruhan sistem asal; peta existing functional components vs incomplete/mock.

C. Jangan menulis semula jika integrasi boleh diguna semula.

D. Tulis `docs/AUDIT.md`, `docs/ARCHITECTURE.md`, `docs/DECISIONS.md`, `docs/FEATURE_BACKLOG.md` serta `docs/QA_MATRIX.md`.

E. Nyatakan integration risks dan pilih implementation approach bagi embedded stream: scrcpy server + decode/render dalam desktop app, atau alternatif sah yang dibuktikan oleh PoC.

F. Semak licenses binary/library sebelum redistribution.

Deliverable: baseline yang boleh dijalankan, audit dengan bukti dan architecture yang dipilih.

STEP 1 — BUKTIKAN EMBEDDED STREAMING POC

A. Sambung satu Android sebenar jika available. Uji `adb devices`, authorization, display dan input.

B. Dapatkan 2 video streams embedded dalam **satu** UI/window dengan per-device source IDs.

C. Bina pipeline capture/transport/decode/render yang mengurus stream lifecycle, backpressure, FPS/resolution, rotation, reconnection dan stream errors.

D. Implement control routing (tap, swipe, type, Home/Back) untuk device tertentu.

E. Benchmark CPU/RAM/FPS/latency dengan angka sebenar (jika hardware ada).

F. Jika satu mekanisme gagal, cari cara yang sah/maintainable dan dokumentasi ADR; jangan settle dengan floating scrcpy windows.

Gate: dua embedded streams + kawalan tanpa salah device. Jika cuma satu telefon tersedia, bina kod dua-stream dan tandakan pengesahan hardware kedua `NOT TESTED`.

STEP 2 — TUKAR KEPADA APLIKASI WINDOWS SEBENAR

A. Gunakan Tauri 2 + React + TypeScript + Tailwind jika sesuai selepas audit.

B. Bina pages: Dashboard, Device Wall, Devices, Pairing, APK & Files, Automation, Virtual Phones (disabled until ready), Activity Logs, Settings.

C. UI premium, dark/light, responsive desktop window, cards, grid, device skeletons, empty states, search/filter dan keyboard shortcuts.

D. Packaging local backend (Python FastAPI jika dikekalkan), ADB/scrcpy binaries, startup health check, clean shutdown, logging.

E. SQLite migrations dan AppData directory standard Windows; no `delete db on schema change`.

Gate: desktop shell stabil dan semua fungsi sedia ada diintegrasi, bukan placeholder palsu.

STEP 3 — MULTI-PHONE DEVICE & APP OPERATIONS

A. Discovery USB, WiFi pairing, authorization state, reconnect/retry, device info.

B. Adapt grid 1/2/4/9, focus mode, low-FPS thumbnail vs high-FPS selected stream.

C. Mouse/keyboard mapping, touch gestures, rotation, text input, clipboard, navigation.

D. APK install/uninstall, file push/pull, screenshots, rename/groups, logs dan progress.

E. Handling disconnected/unauthorized device must not crash others.

F. Add tests for command routing and failure isolation.

Gate: aim 4 physical devices in QA environment, at least 2 simultaneous active streams with independent control; if insufficient phones report device count tested.

STEP 4 — AUTOMATION ENGINE STABIL

A. Build jobs/queue, persistent execution state, safe retries/timeouts/idempotency, cancel/pause/logging.

B. Nodes: launch, tap, swipe, text, wait element (hierarchy first), conditional, loop, APK, file, screenshot, multi-device.

C. UI workflow creation/execution/history with clear status and errors.

D. Avoid AI dependency; cloud LLM only as opt-in later.

E. Tests: success/failure, retries, cancelled workflows, device disconnect, two-device concurrency.

Gate: real or appropriately simulated workflow fully tested, with clear distinction in QA report.

STEP 5 — INSTALLER & WINDOWS RELEASE 1.0

A. Generate `Mirrorhub-Setup-x64.exe` or `.msi` using Tauri official Windows packaging.

B. Bundle runtime/dependencies; user akhir hanya install, buka, sambung telefon.

C. First-run diagnostics, error handling and clear device permissions guidance.

D. Verify clean install, run, close/reopen, DB persistence, update/migration, uninstall on fresh Windows machine/VM if available.

E. Generate versioned build artefact, checksum, release notes, full user guide and troubleshooting.

F. Do not claim Windows installer if build was run only in Linux without verified Windows output.

G. Store final artefact under `dist/` or clear release directory and report exact absolute path.

Gate: INSTALLER REAL + APPLICATION REAL + PHONE MIRRORING REAL. If signing certificate unavailable, create unsigned development/test release with clear warning; do not fake trusted signature.

STEP 6 — LOCAL VIRTUAL ANDROID

A. Detect virtualization/WHPX, available CPU/RAM/SSD.

B. Integrate AVD SDK as permitted; create/start/stop/restart/delete persistent emulator profiles.

C. Detect emulator through ADB and show in same Device Wall.

D. APK install, snapshots, data persistence, resource monitoring, failure recovery.

E. Optional per-emulator VPN with measured egress IP and documented limitations.

F. Tests with 2 instances if sufficient hardware, otherwise mark hardware blocker accurately.

Gate: two Android instances usable and persisted (when hardware supports).

STEP 7 — REMOTE CONNECTOR & CLOUD CONTROL PLANE

A. Refactor shared device/service contracts only as needed; preserve backward-compatible local desktop.

B. Customer auth, org/tenant, connector pairing using short-lived one-time code.

C. Remote stream/control via secure transport (WebRTC with STUN/TURN or justified equivalent).

D. Per-device/session authorization, audit, rate limits, token revocation and no exposed ADB.

E. Two-user cross-tenant negative tests; stage locally via docker where possible.

F. Cloud deployment to an external account only if credentials and approval are available.

Gate: remote access demo + tenant isolation measured, not fabricated.

STEP 8 — CLOUD VIRTUAL ANDROID HOSTS

A. Verify host access to /dev/kvm or suitable virtualization; do not assume OVH VPS supports nested virtualization.

B. Build provisioning scheduler, lifecycle, persistence, snapshots, quotas, isolated runtime/network and storage.

C. User self-service Create Phone → stream → APK → stop/restart → restore.

D. Resource accounting and abuse control.

E. Two-tenant test of isolation and persistence; test real host only if available.

F. Do not rent servers or enable paid infrastructure without approval.

Gate: real virtual Android host, remote usable browser sessions and isolation tests PASS.

STEP 9 — BILLING, LICENSING & COMMERCIAL OPERATIONS

A. Implement Desktop license options without mandatory always-online dependence for core local usage.

B. SaaS account, plan, quota, customer and admin dashboards.

C. Payment gateway adapter (e.g. ToyyibPay), backend verification, webhook idempotency, refund and subscription state.

D. Usage metering, invoices, monitoring/alerts, backup restore, privacy/data deletion and support procedures.

E. Closed beta, load testing, security review and docs.

F. Do not activate live payments or contact real customers before owner explicitly approves.

Gate: end-to-end paid user journey verified in sandbox then production with permission, including failure handling and backup restore.

STRUCTURE SASARAN (ADAPT TO EXISTING REPO)

```
mirrorhub/
  apps/{desktop,web,admin,connector}/
```

```
  services/{local-api,cloud-api,auth,billing,provisioning}/
  packages/{ui,device-core,adb-engine,streaming,automation,virtual-android,api-contracts,
shared}/
  infrastructure/{docker,cloud,monitoring,github-actions}/
  tests/{unit,integration,device,e2e,performance,security}/
  docs/{AUDIT.md,ARCHITECTURE.md,DECISIONS.md,FEATURE_BACKLOG.md,QA_MATRIX.md,USER_GUIDE.md,
RELEASE.md}
```

Do not perform massive monorepo migrations if incremental structure is safer. Database migration must preserve existing users/device entries.

MILESTONE CHECKPOINT & RESUME MECHANISM

Create and maintain these files in repo:

• `MASTER_EXECUTION_STATUS.md` — phase, module status, deliverable path, blocker, next actionable task.

• `docs/QA_MATRIX.md` — PASS/FAIL/NOT TESTED + evidence, date, environment and tested hardware count.

• `docs/DECISIONS.md` — architecture choices and rationale.

• `docs/KNOWN_ISSUES.md` — reproducible defects and workarounds.

• `docs/RELEASE.md` — exact installer path and reproduction instructions.

At the **end of every coding cycle**, update status and continue to next unfinished task. If context window ends, use the files to resume. If an external action is required, write exact blocker and exact action owner must perform, then continue non-blocked work. Avoid repeated questions with known answers.

QUALITY / AUTOMATED VERIFICATION

• Run language-specific formatting/lint, static type checks, unit, integration, device tests, UI E2E and build checks.

• Test stream memory leaks and reconnection; measure FPS, CPU, RAM, bandwidth, latency, crashes; performance targets must derive from actual hardware benchmarking.

• Test app install on fresh Windows; no hidden developer dependencies.

• Security: loopback binding, command/device authorization, input sanitization, dependency scanning, least privilege, no secret logging, no public ADB.

• Test 8-hour stability/soak if actual hardware and execution runtime allow; otherwise mark `NOT TESTED`, do not claim it passed.

• Protect migrations with backups and rollback; no silent deletion.

• Test negative scenarios: USB unplugged, WiFi down, ADB unauthorized, phone restart, emulator fails, broken APK, unavailable network, malformed callback, duplicate webhook.

RESPONSE FORMAT PADA SETIAP MILESTONE

Give brief, factual progress report (in Malay):

```
MILESTONE: [phase / task]
STATUS: PASS / PARTIAL / BLOCKED
CODE CHANGED: [key paths]
TESTS: [commands + results + environment]
USER-VERIFIED OUTPUT: [installer / screenshots / actual phone actions or NOT TESTED]
BLOCKERS: [none or specific external requirement]
NEXT: [next implementation action]
```

Jangan gunakan “semuanya siap” jika installer belum diuji, telefon sebenar belum mirror, atau cloud belum di-deploy. Nyatakan dengan tegas apa yang siap dan apa yang masih perlu bukti.

FINAL ACCEPTANCE CONTRACT

Local release mesti sekurang-kurangnya memenuhi:

1. `.exe`/`.msi` Windows wujud dan boleh dipasang pada PC ujian.
2. Dibuka seperti aplikasi biasa, bukan tab browser atau arahan terminal.
3. Detect minimum 2 telefon real USB/ADB WiFi apabila hardware tersedia.
4. Dua screen embedded dalam **satu aplikasi** dan boleh dikawal secara individu.
5. Connection failures tidak crash seluruh app; reconnect berfungsi.
6. Settings/device data kekal selepas restart; APK/file functions sebenar.
7. QA report dan user guide lengkap; errors tidak disembunyikan.

Full commercial release tambahan mesti memenuhi: local virtual Android, remote connector, cloud tenant isolation, real virtual host, verified billing, security review, monitoring, backups/recovery, customer onboarding.

START NOW — TINDAKAN PERTAMA

1. Jangan balas dengan pelan panjang.
2. Baca repo ini serta fail `CLAUDE.md`/`README.md` terlebih dahulu.
3. Jalankan audit commands dan set baseline project.
4. Mulakan PoC **embedded dua phone** dengan sambungan live jika available.
5. Terus bina desktop shell, fungsi core dan installer Windows sehingga hasil kerja yang boleh digunakan benar-benar wujud.
6. Setiap kali selesai satu task, pilih sendiri task seterusnya berdasarkan dependency dan terus implement/test/fix.
7. Pada akhir sesi, berikan path binary, bukti ujian, status tepat dan *satu* blocker luaran paling penting (jika ada).

Utamakan sistem berfungsi berbanding UI cantik semata-mata. Sasaran pertama yang tidak boleh dikompromi: Mirrorhub Windows local yang boleh install dan mirror beberapa telefon dalam satu dashboard.